

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №3
«Разработка бота для Telegram с использованием языка Go»

Выполнил:
Студент группы ИУ5-31Б
Паронько Д.И.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Задание:

Цель лабораторной работы: изучение разработки ботов в Telegram.

Требования к отчету:

Отчет по лабораторной работе должен содержать:

1. титульный лист;
2. описание задания;
3. текст программы;
4. экранные формы с примерами выполнения программы.

Задание:

1. Разработайте бота для Telegram. Бот должен реализовывать конечный автомат из трех состояний.

Текст программы

bot.go

```
package bot

import (
    "log"
    "unicode"
    "mybot/fsm"
    tgbotapi "github.com/go-telegram-bot-api/telegram-bot-api/v5"
)

type Bot struct {
    botAPI      tgbotapi.BotAPI
    updateCfg   tgbotapi.UpdateConfig
    updateChan  <-chan tgbotapi.Update

    orderMap map[int64][]fsm.UserOrder

    TRANSITION_TABLE []fsm.MessageHandlers
}

func (bot *Bot) Init(token string) {
    botAPIPtr, err := tgbotapi.NewBotAPI(token)

    if err != nil {
        log.Panic("Error: bot init is failed", err)
    }
}
```

```

bot.botAPI = *botAPIPtr
bot.orderMap = make(map[int64][]fsm.UserOrder)

bot.TRANSITION_TABLE = []fsm.MessageHandlers{
    func(order *fsm.UserOrder, msg tgbotapi.Message) {
        bot.HandleNameMessage(order, msg)
    },
    func(order *fsm.UserOrder, msg tgbotapi.Message) {
        bot.HandleAmountMessage(order, msg)
    },
    func(order *fsm.UserOrder, msg tgbotapi.Message) {
        bot.HandleConfirmingMessage(order, msg)
    }
}

func (bot *Bot) SetUpdate(offset int, timeout int) {
    bot.updateCfg = tgbotapi.NewUpdate(offset)
    bot.updateCfg.Timeout = timeout
}

func (bot *Bot) StartListen() {
    bot.updateChan = bot.botAPI.GetUpdatesChan(bot.updateCfg)
}

func (bot *Bot) ProcessOrders() {
    for update := range bot.updateChan {
        userMsg := update.Message
        chatID := update.Message.Chat.ID

        if userMsg == nil {
            continue
        }

        if userMsg.IsCommand() && userMsg.Command() == "start" {
            newOrder := fsm.UserOrder{CurState: fsm.WAITING_NAME}

            bot.orderMap[chatID] = append(bot.orderMap[chatID], newOrder)
            nameQuery := "Enter the name of good"

            botMsg := tgbotapi.NewMessage(userMsg.Chat.ID, nameQuery)
            botMsg.ReplyToMessageID = userMsg.MessageID

            _, err := bot.botAPI.Send(botMsg)
            if err != nil {
                log.Printf("Ошибка при отправке сообщения: %v", err)
            }

            continue
        }
    }
}

```

```

usrOrderList := bot.orderMap[chatID]
usrLastOrder := usrOrderList[len(usrOrderList)-1]

if usrLastOrder.CurState == fsm.SUCCESS {
    nameQuery := "Your order is done. Write command /start for new order"

    botMsg := tgbotapi.NewMessage(userMsg.Chat.ID, nameQuery)
    botMsg.ReplyToMessageID = userMsg.MessageID
    bot.botAPI.Send(botMsg)
    continue
}

log.Println(usrLastOrder.CurState, len(usrOrderList))
bot.TRANSITION_TABLE[usrLastOrder.CurState](&usrLastOrder, *userMsg)

bot.orderMap[chatID][len(usrOrderList)-1] = usrLastOrder
}
}

func (bot *Bot) HandleConfirmingMessage(order *fsm.UserOrder, msg
tgbotapi.Message) {
    if msg.Text != "Y" && msg.Text != "N" {
        return
    }

    order.IsConfirm = msg.Text == "Y"
    order.CurState = fsm.SUCCESS

    botMsg := tgbotapi.NewMessage(msg.Chat.ID, "Order is done")
    botMsg.ReplyToMessageID = msg.MessageID

    _, err := bot.botAPI.Send(botMsg)
    if err != nil {
        log.Printf("Ошибка при отправке сообщения: %v", err)
    }
}

func (bot *Bot) HandleAmountMessage(order *fsm.UserOrder, msg tgbotapi.Message) {
    log.Printf("В функции запроса суммы")

    if len(msg.Text) == 0 {
        return
    }

    for _, ch := range msg.Text {
        if !unicode.IsDigit(ch) {
            return
        }
    }

    order.CurState = fsm.CONFIRMING

```

```

confirmQuery := "Confirm? [Y/N]"
botMsg := tgbotapi.NewMessage(msg.Chat.ID, confirmQuery)
botMsg.ReplyToMessageID = msg.MessageID

_, err := bot.botAPI.Send(botMsg)
if err != nil {
    log.Printf("Ошибка при отправке сообщения: %v", err)
}

order.Cost = msg.Text
}

func (bot *Bot) HandleNameMessage(order *fsm.UserOrder, msg tgbotapi.Message) {
    if len(msg.Text) == 0 {
        return
    }

    for _, ch := range msg.Text {
        if !unicode.IsLetter(ch) && !unicode.IsDigit(ch) {
            return
        }

        if unicode.IsLetter(ch) {
            order.CurState = fsm.WAITING_AMOUNT
        }
    }

    if order.CurState != fsm.WAITING_AMOUNT {
        return
    }

    amountQuery := "Enter the amount"
    botMsg := tgbotapi.NewMessage(msg.Chat.ID, amountQuery)
    botMsg.ReplyToMessageID = msg.MessageID

    _, err := bot.botAPI.Send(botMsg)
    if err != nil {
        log.Printf("Ошибка при отправке сообщения: %v", err)
    }

    order.GoodName = msg.Text
}

```

fsm.go

```

package fsm

import (
    tgbotapi "github.com/go-telegram-bot-api/telegram-bot-api/v5"

```

```

)

type State int

const (
    WAITING_NAME State = iota
    WAITING_AMOUNT
    CONFIRMING
    SUCCESS
)

type UserOrder struct {
    GoodName string
    Cost      string
    IsConfirm bool
    CurState  State
}

type MessageHandlers func(order *UserOrder, msg tgbotapi.Message)

```

main.go

```

package main

import (
    "mybot/bot"
)

func main() {
    var myBotToken string = "8211768908:AAFqZ4zmcI-W6BsKjFKpNC3rTAOM3emzvK4"
    var myBot bot.Bot
    myBot.Init(myBotToken)
    myBot.SetUpdate(0, 60)
    myBot.StartListen()
    myBot.ProcessOrders()
}

```

Результат выполнения

